

Patroni Cluster Management

About

Here the link to the video of the runbook simulation.

Patroni is an open source project for providing automatic HA solution for PostgreSQL databases. It also has centralized configuration management capabilities and it is fully integrated with Consul to provide *consensus*, and dns services. Each patroni instance has a **consul** agent, who acts as a proxy for the GitLab Consul fleet.

Patroni binaries run in the same host as the PostgreSQL database. In a sense, Patroni can be seen as a way of managing a PostgreSQL instance. Configuration and general operations of PostgreSQL instances are now achieved by using the corresponding Patroni command to do so.

Patroni basics

Patroni configuration file

Is located at `/var/opt/gitlab/patroni/patroni.yml`. It contains all the directives to configure patroni and PostgreSQL instance too. It is divided into sections, and the main ones are:

- **scope**: Cluster name
- **name**: Name of this instance
- **consul**: Location of the consul agent (usually localhost)
- **bootstrap**: Directives for configuring and deploying new instances
- **postgresql**: General configuration settings for PostgreSQL

Checking service status

The **patroni** service is managed with **systemd**, so you can check the service status with `systemctl status patroni` and logs with `journalctl -u patroni` (it should be enabled and running).

Run `gitlab-patronictl list` to check the state of the patroni cluster, you should see the new node join the cluster and go through the following states:

- creating replica
- starting
- running

the node will also be added to the consul DNS entry, you can verify that with:

```
dig @127.0.0.1 -p8600 +short replica.patroni.service.consul.
```

At the moment of writing the database is ~7TB big and it takes ~4h for a new node to catch up.

Cluster information

Run `gitlab-patronictl list` on any Patroni member to list all the cluster members and their statuses. There can be only a single **Leader** for the cluster.

A leader change is an important event. When unexpected, this leader change is call a **FailOver**. Under controlled circumstances (i.e. when upgrading), this is call a **SwitchOver**, and Patroni has a special command for doing this.

```
patroni-01-db-gstg $ gitlab-patronictl list
```

Cluster	Member	Host	Role
pg11-ha-cluster	patroni-01-db-gstg.c.gitlab-staging-1.internal	10.224.29.101	
pg11-ha-cluster	patroni-02-db-gstg.c.gitlab-staging-1.internal	10.224.29.102	
pg11-ha-cluster	patroni-04-db-gstg.c.gitlab-staging-1.internal	10.224.29.104	Leader
pg11-ha-cluster	patroni-05-db-gstg.c.gitlab-staging-1.internal	10.224.29.105	
pg11-ha-cluster	patroni-06-db-gstg.c.gitlab-staging-1.internal	10.224.29.106	

You may want to use it in conjunction with `watch`, to automatically refresh the command each 30 seconds or so:

Cluster	Member	Host	Role
pg11-ha-cluster	patroni-01-db-gstg.c.gitlab-staging-1.internal	10.224.29.101	
pg11-ha-cluster	patroni-02-db-gstg.c.gitlab-staging-1.internal	10.224.29.102	
pg11-ha-cluster	patroni-04-db-gstg.c.gitlab-staging-1.internal	10.224.29.104	Leader
pg11-ha-cluster	patroni-05-db-gstg.c.gitlab-staging-1.internal	10.224.29.105	
pg11-ha-cluster	patroni-06-db-gstg.c.gitlab-staging-1.internal	10.224.29.106	

You may want to use it in conjunction with `watch`, to automatically refresh the command each 30 seconds or so:

```
watch -n 30 'gitlab-patronictl list'
```

Under certain circumstances, you may see an extra column labeled **Pending restart**, and look like this:

Cluster	Member	Host	Role	
pg11-ha-cluster	patroni-01-db-gstg.c.gitlab-staging-1.internal	10.224.29.101		
pg11-ha-cluster	patroni-02-db-gstg.c.gitlab-staging-1.internal	10.224.29.102		
pg11-ha-cluster	patroni-04-db-gstg.c.gitlab-staging-1.internal	10.224.29.104	Leader	
pg11-ha-cluster	patroni-05-db-gstg.c.gitlab-staging-1.internal	10.224.29.105		

```
| pg11-ha-cluster | patroni-06-db-gstg.c.gitlab-staging-1.internal | 10.224.29.106 |
+-----+-----+-----+-----+
watch -n 30 'gitlab-patronictl list'
```

Under certain circumstances, you may see an extra column labeled `Pending restart`, which

```
```sh
+-----+-----+-----+-----+
| Cluster | Member | Host | Role |
+-----+-----+-----+-----+
| pg11-ha-cluster | patroni-01-db-gstg.c.gitlab-staging-1.internal | 10.224.29.101 | |
| pg11-ha-cluster | patroni-02-db-gstg.c.gitlab-staging-1.internal | 10.224.29.102 |
| pg11-ha-cluster | patroni-04-db-gstg.c.gitlab-staging-1.internal | 10.224.29.104 | Leader |
| pg11-ha-cluster | patroni-05-db-gstg.c.gitlab-staging-1.internal | 10.224.29.105 |
| pg11-ha-cluster | patroni-06-db-gstg.c.gitlab-staging-1.internal | 10.224.29.106 |
+-----+-----+-----+-----+
```

In the previous example, members `patroni-02-db-gstg.c.gitlab-staging-1.internal` and `patroni-03-db-gstg.c.gitlab-staging-1.internal` are waiting for a restart in order to apply some configuration changes.

If you need to know what those *Pending restart* settings are, execute the following in the instance you need to verify:

```
sudo gitlab-psql -c "select name, setting, short_desc, sourcefile, sourceline from pg_settings"
```

And a possible result:

```
name | setting | short_desc |
-----+-----+-----+
max_connections | 500 | Sets the maximum number of concurrent connections. | /etc/postgresql/
```

### Restarting a node

To force the restart of a specific member you can execute (on any node) `sudo gitlab-patronictl restart pg11-ha-cluster <member>`

In the example above, members `patroni-02-db-gstg.c.gitlab-staging-1.internal` and `patroni-03-db-gstg.c.gitlab-staging-1.internal` are waiting for a restart in order to apply some configuration changes.

If you need to know which those *Pending restart* settings are, just need to execute

```
sudo gitlab-psql -c "select name, setting, short_desc, sourcefile, sourceline from pg_settings"
```

And a possible result:

```
name | setting | short_desc |
-----+-----+-----+
max_connections | 500 | Sets the maximum number of concurrent connections. | /etc/postgresql/
```

## Bootstrapping modes

### Normal bootstrapping

Normal bootstrapping is when you start a brand-new cluster with zero data. Patroni will create the PostgreSQL database cluster using `initdb` with the options specified in `node['gitlab-patroni']['patroni']['config']['bootstrap']['initdb']`.

**Creating base configuration for normal bootstrapping** By default our Chef cookbooks won't create the `postgresql.base.conf` file, to prevent Patroni refusing to start due to a non-empty PGDATA folder. When creating a new cluster from scratch, you can signal the Chef client run to provision this file by simply touching it. For example:

```
sudo -u gitlab-psql touch /var/opt/gitlab/postgresql/data/postgresql.base.conf
```

Where `/var/opt/gitlab/postgresql/data/` corresponds to `node['gitlab-patroni']['postgresql']['conf`

### Standby bootstrapping

Standby bootstrapping is starting a Patroni cluster that replicates from a remote master (i.e. not part of the Patroni cluster).

You need to specify the following Chef attributes to start a cluster in standby mode:

```
node['gitlab-patroni']['patroni']['config']['bootstrap']['dcs']['standby_cluster'] = {
 "host": "remote.host.com",
 "port": "5432",
 "primary_slot_name": "patroni_repl_slot"
}
```

You'd need some extra setup on the remote master side:

1. Create a replication user with a username/password matching the ones you have in `node['gitlab-patroni']['patroni']['users']['replication']`
  - `CREATE USER "gitlab-replicator" LOGIN REPLICATION PASSWORD 'hunter1';`
2. Create a superuser with a username/password matching the ones you have in `node['gitlab-patroni']['patroni']['users']['superuser']`
  - `CREATE USER "gitlab-superuser" LOGIN SUPERUSER REPLICATION PASSWORD 'hunter1';`
3. Create a physical replication slot with the name you specified in `primary_slot_name` above
  - `SELECT * FROM pg_create_physical_replication_slot("patroni_repl_slot");`
4. Allow the replication user into the remote master through `pg_hba` entries
  - `host replication gitlab-replicator 10.0.0.0/8 md5`
  - `host replication gitlab-replicator 127.0.0.1/32 md5`

## Configuring PostgreSQL

You can specify any PostgreSQL parameter under `node['gitlab-patroni']['postgresql']['parameters']`, except for the following parameters:

- `cluster_name`
- `wal_level`
- `hot_standby`
- `max_connections`
- `max_wal_senders`
- `wal_keep_segments`
- `max_prepared_transactions`
- `max_locks_per_transaction`
- `track_commit_timestamp`
- `max_replication_slots`
- `max_worker_processes`
- `wal_log_hints`

These parameters are specifically handled by Patroni for replication purposes. Some of them can't be changed (like `wal_log_hints`), those that can be changed need to be specified under `node['gitlab-patroni']['patroni']['config']['bootstrap']['dcs']['postgresql']`.

While nothing prevents you from specifying them under `node['gitlab-patroni']['postgresql']['parameters']`, it will likely confuse Patroni into thinking that the cluster needs a restart (you may see a “Pending Restart” column when running `gitlab-patronictl list`).

When parameters are updated in Chef and propagated across the cluster, Patroni updates `postgresql.conf` then signals PostgreSQL to reload the configuration. A restart may still be needed for some parameters, which you can see hints of in the logs, so you may need to run `gitlab-patronictl restart pg11-ha-cluster MEMBER_NAME`.

## Pausing Patroni

Quoting Patroni docs:

Under certain circumstances Patroni needs to temporary step down from managing the cluster, while still retaining the cluster state in DCS. Possible use cases are uncommon activities on the cluster, such as major version upgrades or corruption recovery. During those activities nodes are often started and stopped for the reason unknown to Patroni, some nodes can be even temporary promoted, violating the assumption of running only one master. Therefore, Patroni needs to be able to “detach” from the running cluster, implementing an equivalent of the maintenance mode in Pacemaker.

Pausing the cluster disables automatic failover. This is desirable when doing tasks such as upgrading Consul, or during a maintenance of the whole Patroni

cluster.

Note that disabling automatic failover can have undesirable side-effects, for example, if the primary PostgreSQL went down for any reason while Patroni is paused, there will be no primary for the clients to write to, effectively resulting in 500 errors for the end-user, so plan ahead carefully.

It is important to disable `chef-client` before pausing the cluster, otherwise a regular `chef-client` can revert the pause status.

To pause the cluster, run the following commands:

```
workstation $ knife ssh roles:<env>-base-db-patroni 'sudo chef-client-disable "Database"'
patroni-01-db-gstg $ sudo gitlab-patronictl pause --wait pg11-ha-cluster
```

You can verify the result of pausing the cluster by running:

```
patroni-01-db-gstg $ sudo gitlab-patronictl list | grep 'Maintenance mode'
```

Run these commands to unpause/resume the cluster:

```
patroni-01-db-gstg $ sudo gitlab-patronictl resume --wait pg11-ha-cluster
workstation $ knife ssh roles:<env>-base-db-patroni 'sudo chef-client-enable'
```

## Restarting Patroni When Paused

When the cluster is paused, before restarting the Patroni process, it is better to check if Postgres postmaster didn't start when the system clock was skewed for any reason:

```
patroni-01-db-gstg # postmaster=/var/opt/gitlab/postgresql/data11/postmaster.pid;\
postpid=$(cat $postmaster | head -1);\
posttime=$(cat $postmaster | tail -n +3 | head -1);\
btime=$(cat /proc/stat | grep btime | cut -d' ' -f2);\
starttime=$(cat /proc/$postpid/stat | awk '{print $22}');\
clktck=$(getconf CLK_TCK);\
echo $(((starttime / $clktck + $btime) - $posttime))
```

If the command above returned a value higher than 3, then Patroni is going to have trouble starting, so it is advised to fix this issue with the help of a DBRE before restarting.

## Upgrading Patroni

Patroni version is controlled by Chef, upgrading it should be as simple as changing an attribute in a role or a cookbook. Since Patroni would need to be restarted (and subsequently, PostgreSQL), careful execution of the change is needed to avoid database errors on the client side. While pausing Patroni (see relevant section above) may be employed to restart Patroni without disturbing PostgreSQL, it's not recommended to go this route as converging Chef can undo the pausing action, which can introduce unintended results.

Instead, we recommend, one at a time, putting replicas into maintenance mode (see relevant section below), upgrading Patroni through Chef, then putting replicas out of maintenance. For the primary, we initiate a switchover to one of the upgraded replicas then we upgraded it once it's been demoted.

The exact sequence of upgrading replicas has been encapsulated into an Ansible playbook. The playbook expects an MR in the chef-repo project to be specified in `variables.yml` under the target environment, and an API token to be used to merge such MR.

The playbook can be run as follows:

```
$ git clone git@gitlab.com:gitlab-com/gl-infra/ansible-migrations.git
$ cd ansible-migrations
Change relevant MRs in variables.yml
$ tmux
$ OPS_API_TOKEN=secure-token MIGRATION_ENV=gprd-or-gstg ansible-playbook -i production-1172/
```

## Replica Maintenance

If clients are connecting to replicas by means of service discovery (as opposed to hard-coded list of hosts), you can temporarily remove a replica from the list of hosts used by the clients by tagging it as not suitable for failing over (`nofailover: true`) and load balancing (`noloadbalance: true`).

1. `sudo chef-client-disable "Database maintenance issue prod#xyz"`
2. Add a `tags` section to `/var/opt/gitlab/patroni/patroni.yml` on the node:

```
tags:
 nofailover: true
 noloadbalance: true
```

3. `sudo systemctl reload patroni`
4. Test the efficacy of that reload by checking for the node name in the list of replicas:

```
dig @127.0.0.1 -p 8600 db-replica.service.consul. SRV
```

If the name is absent, then the reload worked.

5. Wait until all client connections are drained from the replica (it depends on the interval value set for the clients), use this command to track number of client connections:

```
while true; do for c in /usr/local/bin/pgb-console*; do sudo $c -c 'SHOW CLIENTS;'; do
```

Note: Usually there are three pgbouncer instances running on a single replica.

You can see an example of taking a node out of service in this issue.

## Caveat

If the maintenance involves restarting/stopping either the replica host or Consul agent itself, consider removing the `db-replica` service(s) from Consul before attempting such restart/stop. Failing to do so could result in the replica being added back to Rails DB load-balancing, for a brief time, which in the case of stopping the node, could result in a spike of client errors as they wouldn't be able to connect to the stopped node.

Removing the service from Consul can be done as follows:

```
sudo rm /etc/consul/conf.d/db-replica*
sudo systemctl reload consul
```

Now it should be safe to restart/stop either the node or Consul agent.

For more information on why this is necessary, please read this investigation.

An example for stopping, decommissioning then re-provisioning a replica can be found in this issue.

## Long-term marking a replica as in maintenance

The method described above is suitable for short-term maintenance (e.g. 1 hour or less), during which it's acceptable to keep `chef-client` disabled. However, if the replica is expected to be in maintenance for many hours or days, it is undesirable to keep `chef-client` disabled during such period.

Using node attributes to set such attributes is not ideal as node attributes are not as visible as roles, and Chef nodes are removed when its GCP node is restarted, so it's not totally persistent.

Alternatively, we can utilize a special Chef role (`<env>-base-db-patroni-maintenance`) that sets `nofailover` and `noloadbalance` to `true`, and then instruct Terraform to assign it to a particular instance.

1. In Chef repo, add the role `<env>-base-db-patroni-maintenance` into the node `run_list` and then execute `chef-client` on the specific node:

```
knife node run_list add <node-fqdn> "role[<env>-base-db-patroni-maintenance]"
ssh <node-fqdn> "sudo chef-client"
```

2. Verify the replica is in maintenance by checking for the node name in the list of replicas:

```
dig @127.0.0.1 -p 8600 db-replica.service.consul. SRV
```

If the name is absent, then the reload worked.

3. Check Grafana `pgbouncer_stats_queries_pooled_total` metric



4. To let the configuration in Sync add the `${var.environment}-base-db-patroni-maintenance` role in the `chef_run_list_extra` for the specific patroni module and node, like the following snippet:

```
nodes = {
 ...
 10 = {
 chef_run_list_extra = "\"role[${var.environment}-base-db-patroni-maintenance]\""
 }
}
```

Replace 10 with the zero-based index of the replica we are targeting.

5. Apply the Terraform change.

**Remove long-term maintenance from a Replica** These are the steps to revert the long-term marking of a replica as in maintenance:

1. In Chef repo, remove the role `<env>-base-db-patroni-maintenance` from the node `run_list` and then execute `chef-client` on the specific node:

```
knife node run_list remove <node-fqdn> "role[<env>-base-db-patroni-maintenance]"
ssh <node-fqdn> "sudo chef-client"
```

2. Don't forget to revert the MR you created to add `${var.environment}-base-db-patroni-maintenance` in the node `chef_run_list_extra`

### Legacy Method (Consul Maintenance)

*:warning: This method only works if the clients are configured with a `replica.patroni.service.consul` DNS record, it won't work properly if they are configured with `db-replica.service.consul` record. Check `/var/opt/gitlab/gitlab-rails/etc/database.yml` before you proceed.*

In the past we have sometimes used consul directly to remove the replica from the replica DNS entry (bear in mind this does not prevent the node from becoming the primary).

```
patroni-01-db-gstg $ consul maint -enable -service=patroni-replica -reason="Production issue"
```

You can verify the action by running:

```
patroni-01-db-gstg $ dig @127.0.0.1 -p8600 +short replica.patroni.service.consul. | grep $(hostname)
```

Wait until all client connections are drained from the replica (it depends on the interval value set for the clients), use this command to track number of client connections:

```
patroni-01-db-gstg $ while true; do for c in /usr/local/bin/pgb-console*; do sudo $c -c 'SHOW CONNECTIONS'; done; sleep 5; done
```

After you're done with the maintenance, disable Consul service maintenance and verify it:

```
patroni-01-db-gstg $ consul maint -disable -service=patroni-replica
patroni-01-db-gstg $ dig @127.0.0.1 -p8600 +short replica.patroni.service.consul. | grep $(
```

## Failover/Switchover

Failover and Switchover are similar in their end-result, still there are slight differences between them:

- You can't do a switchover when the cluster has no leader
- Switchover can be scheduled to happen in a later time
- You need to specify a member to failover to, switchover does not and it will choose one at random.

That said, you can initiate any of them using `gitlab-patronictl switchover` or `gitlab-patronictl failover` and entering values when prompted.

## Problems with replication after failover

Sometimes, after a failover, the old primary's timeline will have continued and diverged from the new primary's timeline. Patroni will automatically attempt to `pg_rewind` the timeline of the old primary to a point at which it can begin replicating from the new primary, becoming healthy again. We have occasionally seen this fail, for example with a statement timeout.

If for whatever reason you can't get the node to a healthy state and don't mind waiting several hours, you can reinitialise the node:

```
root@pg$ gitlab-patronictl reinit pg11-ha-cluster patroni-XX-db-gprd.c.gitlab-production.int
```

This command can be run from any member of the patroni cluster. It wipes the data directory, takes a `pg_basebackup` from the new primary, and begins replicating again.

## Problems with Performance after failover or switchover

When a switchover or failover happens the new leader will always be out of date with analytics and as result, we will have queries performing poorly. Therefore, the next step is to update these analytics to optimize the execution query plan by running an `ANALYZE`. Postgres `ANALYZE` runs sequential, but we can run `vacuumdb`, a wrapper around the SQL command `VACUUM`, and can run in parallel, with options:

`--analyze-only` that only calculate statistics for use by the optimizer without vacuum.

`--jobs=N` will execute the analyze commands in parallel by running N jobs commands simultaneously.

The full command is:

```
vacuumdb --analyze-only --jobs=N
```

### Diverged timeline WAL segments in GCS after failover

Our primary Postgres node is configured to archive WAL segments to GCS. These segments are pulled by wal-e on another node in recovery mode, and replayed. This process acts as a continuous test of our ability to restore our database state from archived WAL segments. Sometimes, during a failover, both the old master and the new will have uploaded WAL segments, causing the DR archive that is consuming these segments from GCS to not be able to replay the diverged timeline. In the past we have solved this by rolling back the DR archive to an earlier state:

1. In the GCE console: stop the machine
2. Edit the machine: write down (or take a screenshot) of the attachment details of **all** extra disks. Specifically, we want the custom name (if any) and the order they are attached in.
3. Detach the data disk and save the machine.
4. In the GCE console, find the most recent snapshot of the data disk before the incident occurred. Copy its ID.
5. Find the data disk in GCE. Write down its name, zone, type (standard/SSD), and labels.
6. Delete the data disk.
7. Create a new GCE disk with the same name, zone, and type as the old data disk. Select the “snapshot” option as source and enter the snapshot ID.
8. When the disk has finished creating, attach it to the stopped machine using the GCE console.
9. Save the machine and examine the order of attached disks. If they are not in the same order as before, you will have to detach and reattach disks as appropriate. This is necessary because unfortunately we still have code that makes assumptions about the udev-ordering of disks (sdb, sdc etc).
10. Start the machine.
11. **ssh** to the machine and start postgres: `gitlab-ctl start postgresql`.
12. Tail the log file at `/var/log/gitlab/postgresql/current`. You should see it successfully ingesting WAL segments in sequential order, e.g.: `LOG: restored log file "00000017000128AC000000087" from archive`.
13. You should also see a message “FATAL: the database system is starting up” every 15s. These are due to attempted scrapes by the postgres exporter.

After a few minutes, these messages should stop and metrics from the machine should be observable again.

14. In prometheus, you should see the `pg_replication_lag` metric for this instance begin to decrease. Recovery from GCS WAL segments is slow, and during times of high traffic (when the postgres data ingestion rate is high) recovery will slow. It might take days to recover, so be sure to silence any replication lag alerts for enough time not to rudely wake the on-call.
15. Check there is no terraform plan diff for the archival replicas. Run the following for the `gprd` environment:

```
tf plan -out plan -target module.postgres-dr-archive -target module.postgres-dr-delayed
```

If there is a plan diff for mutable things like labels, apply it. If there is a plan diff for more severe things like disk name, you might have made a mistake and will have to repeat this whole procedure.

This procedure is rather manual and lengthy, but this does not happen often and has no directly user-facing impact.

## Replacing a cluster node

The process and steps to diagnose and replace an unhealthy Patroni node are detailed in the [Handling Unhealthy Patroni Replica](#) runbook.

## Scaling the cluster up

Here the link to the [Scale Up Patroni](#) runbook.

## Scaling the cluster down

Here the link to the [Scale Down Patroni](#) runbook.

## Replacing the whole cluster (with a new one)

**Take care when doing these steps, results can be catastrophic**

In case there's a need to replace a current cluster with a new one, say, for testing purposes or replication from a remote cluster got messed-up, you can remove the current cluster without the need to destroy and re-create the node.

```
chef-repo $ knife ssh roles:gstg-base-db-patroni 'sudo systemctl stop patroni'
chef-repo $ knife ssh roles:gstg-base-db-patroni 'sudo rm -rf /var/opt/gitlab/postgresql/data'
chef-repo $ knife ssh roles:gstg-base-db-patroni 'consul kv delete -recurse service/pg11-ha'
chef-repo $ knife ssh roles:gstg-base-db-patroni 'sudo systemctl start patroni'
```

You may need to adjust the Patroni Chef role before restarting the `patroni` service, like adding the standby config followed by running `sudo chef-client` across the cluster.

## Auditing patroni

Patroni holds a log file under `/var/log/gitlab/patroni/patroni.log` where patroni activity and messages can be found:

```
gerardoherzig@patroni-03-db-gstg.c.gitlab-staging-1.internal:~$ sudo tail /var/log/gitlab/patroni/patroni.log
2020-06-26_19:08:55 patroni-03-db-gstg patroni[14726]: 2020-06-26 19:08:55,397 INFO: no act
2020-06-26_19:09:05 patroni-03-db-gstg patroni[14726]: 2020-06-26 19:09:05,278 INFO: Lock d
2020-06-26_19:09:05 patroni-03-db-gstg patroni[14726]: 2020-06-26 19:09:05,278 INFO: does r
2020-06-26_19:09:05 patroni-03-db-gstg patroni[14726]: 2020-06-26 19:09:05,288 INFO: no act
2020-06-26_19:09:15 patroni-03-db-gstg patroni[14726]: 2020-06-26 19:09:15,279 INFO: Lock d
2020-06-26_19:09:15 patroni-03-db-gstg patroni[14726]: 2020-06-26 19:09:15,279 INFO: does r
2020-06-26_19:09:15 patroni-03-db-gstg patroni[14726]: 2020-06-26 19:09:15,290 INFO: no act
2020-06-26_19:09:25 patroni-03-db-gstg patroni[14726]: 2020-06-26 19:09:25,394 INFO: Lock d
2020-06-26_19:09:25 patroni-03-db-gstg patroni[14726]: 2020-06-26 19:09:25,394 INFO: does r
2020-06-26_19:09:25 patroni-03-db-gstg patroni[14726]: 2020-06-26 19:09:25,405 INFO: no act
```

This is mainly useful when investigating an event like an unexpected leader change (failover)